16. Architecture and Requirements

The two most important requirements for major success are: first, being in the right place at the right time, and second, doing something about it.
—Ray Kroc

Architectures exist to build systems that satisfy requirements. That's obvious. What may be less obvious is that, to an architect, not all requirements are created equal. Some have a much more profound effect on the architecture than others. An architecturally significant requirement (ASR) is a requirement that will have a profound effect on the architecture—that is, the architecture might well be dramatically different in the absence of such a requirement.

You cannot hope to design a successful architecture if you do not know the ASRs. ASRs often, but not always, take the form of quality attribute requirements—the performance, security, modifiability, availability, usability, and so forth, that the architecture must provide to the system. In Chapters 5–13 we introduced patterns and tactics to achieve quality attributes. Each time you select a pattern or tactic to use in your architecture, you are changing the architecture as a result of the need to meet quality attribute requirements. The more difficult and important the QA requirement, the more likely it is to significantly affect the architecture, and hence to be an ASR.

Architects have to identify ASRs, usually after doing a significant bit of work to uncover candidate ASRs. Competent architects know this, and as we observe experienced architects going about their duties, we notice that the first thing they do is start talking to the important stakeholders. They're gathering the information they need to produce the architecture that will respond to the project's needs—whether or not this information has already been identified.

This chapter provides some systematic means for identifying the ASRs and other factors that will shape the architecture.

16.1. GATHERING ASRS FROM REQUIREMENTS DOCUMENTS

An obvious location to look for candidate ASRs is in the requirements documents or in user stories. After all, we are looking for requirements, and requirements should be in requirements documents. Unfortunately, this is not usually the case, although as we will see, there is information in the requirements documents that can be of use.

Don't Get Your Hopes Up

Many projects don't create or maintain the kind of requirements document that professors in software engineering classes or authors of traditional software engineering books love to prescribe. Whether requirements are specified using the "MoSCoW" style (must, should, could, won't), or as a collection of "user stories," neither of these is much help in nailing down quality attributes.

Furthermore, no architect just sits and waits until the requirements are "finished" before starting work. The architect *must* begin while the requirements are still in

flux. Consequently, the QA requirements are quite likely to be up in the air when the architect starts work. Even where they exist and are stable, requirements documents often fail an architect in two ways.

First, most of what is in a requirements specification does not affect the architecture. As we've seen over and over, architectures are mostly driven or "shaped" by quality attribute requirements. These determine and constrain the most important architectural decisions. And yet the vast bulk of most requirements specifications is focused on the required features and functionality of a system, which shape the architecture the least. The best software engineering practices do prescribe capturing quality attribute requirements. For example, the Software Engineering Body of Knowledge (SWEBOK) says that quality attribute requirements are like any other requirements. They must be captured if they are important, and they should be specified unambiguously and be testable.

In practice, though, we rarely see adequate capture of quality attribute requirements. How many times have you seen a requirement of the form "The system shall be modular" or "The system shall exhibit high usability" or "The system shall meet users' performance expectations"? These are not requirements, but in the best case they are invitations for the architect to begin a conversation about what the requirements in these areas really are.

Second, much of what is useful to an architect is not in even the best requirements document. Many concerns that drive an architecture do not manifest themselves at all as observables in the system being specified, and so are not the subject of requirements specifications. ASRs often derive from business goals in the development organization itself; we'll explore this in Section 16.3. Developmental qualities are also out of scope; you will rarely see a requirements document that describes teaming assumptions, for example. In an acquisition context, the requirements document represents the interests of the acquirer, not that of the developer. But as we saw in Chapter 3, stakeholders, the technical environment, and the organization itself all play a role in influencing architectures.

Sniffing Out ASRs from a Requirements Document

Although requirements documents won't tell an architect the whole story, they are an important source of ASRs. Of course, ASRs aren't going to be conveniently labeled as such; the architect is going to have to perform a bit of excavation and archaeology to ferret them out.

Chapter 4 categorizes the design decisions that architects have to make. Table 16.1 summarizes each category of architectural design decision, and it gives a list of requirements to look for that might affect that kind of decision. If a requirement affects the making of a critical architectural design decision, it is by definition an ASR.

Table 16.1. Early Design Decisions and Requirements That Can Affect Them

Design Decision Category	Look for Requirements Addressing . . .
Allocation of Responsibilities	Planned evolution of responsibilities, user roles, system modes, major processing steps, commercial packages
Coordination Model	Properties of the coordination (timeliness, currency, completeness, correctness, and consistency) Names of external elements, protocols, sensors or actuators (devices), middleware, network configurations (including their security properties) Evolution requirements on the list above
Data Model	Processing steps, information flows, major domain entities, access rights, persistence, evolution requirements
Management of Resources	Time, concurrency, memory footprint, scheduling, multiple users, multiple activities, devices, energy usage, soft resources (buffers, queues, etc.) Scalability requirements on the list above
Mapping among Architectural Elements	Plans for teaming, processors, families of processors, evolution of processors, network configurations
Binding Time Decisions	Extension of or flexibility of functionality, regional distinctions, language distinctions, portability, calibrations, configurations
Choice of Technology	Named technologies, changes to technologies (planned and unplanned)

16.2. GATHERING ASRS BY INTERVIEWING STAKEHOLDERS

Say your project isn't producing a comprehensive requirements document. Or it is, but it's not going to have the QAs nailed down by the time you need to start your design work. What do you do?

Architects are often called upon to help set the quality attribute requirements for a system. Projects that recognize this and encourage it are much more likely to be successful than those that don't. Relish the opportunity. Stakeholders often have no idea what QAs they want in a system, and no amount of nagging is going to suddenly instill the necessary insight. If you insist on quantitative QA requirements, you're likely to get numbers that are arbitrary, and there's a good chance that you'll find at least some of those requirements will be very difficult to satisfy.

Architects often have very good ideas about what QAs are exhibited by similar systems, and what QAs are reasonable (and reasonably straightforward) to provide. Architects can usually provide quick feedback as to which quality attributes are going to be straightforward to achieve and which are going to be problematic or even prohibitive. And architects are the only people in the room who can say, "I can actually deliver an architecture that will do better than what you had in mind—would that be useful to you?"

Interviewing the relevant stakeholders is the surest way to learn what they know and need. Once again, it behooves a project to capture this critical information in a systematic, clear, and repeatable way. Gathering this information from stakeholders can be achieved by many methods. One such method is the Quality Attribute Workshop (QAW), described in the sidebar.

The results of stakeholder interviews should include a list of architectural drivers and a set of QA scenarios that the stakeholders (as a group) prioritized. This information can be used to do the following:

- Refine system and software requirements
- Understand and clarify the system's architectural drivers
- Provide rationale for why the architect subsequently made certain design decisions
- Guide the development of prototypes and simulations
- Influence the order in which the architecture is developed

The Quality Attribute Workshop

The QAW is a facilitated, stakeholder-focused method to generate, prioritize, and refine quality attribute scenarios before the software architecture is completed. The QAW is focused on system-level concerns and specifically the role that software will play in the system. The QAW is keenly dependent on the participation of system stakeholders.[1]

The QAW involves the following steps:

Step 1. QAW Presentation and Introductions. QAW facilitators describe the motivation for the QAW and explain each step of the method. Everyone introduces themselves, briefly stating their background, their role in the organization, and their relationship to the system being built.

Step 2. Business/Mission Presentation. The stakeholder representing the business concerns behind the system (typically a manager or management representative) spends about one hour presenting the system's business context, broad functional requirements, constraints, and known quality attribute requirements. The quality attributes that will be refined in later steps will be derived largely from the business/mission needs presented in this step.

Step 3. Architectural Plan Presentation. Although a detailed system or software architecture might not exist, it is possible that broad system descriptions, context drawings, or other artifacts have been created that describe some of the system's technical details. At this point in the workshop, the architect will present the system architectural plans as they stand. This lets stakeholders know the current architectural thinking, to the extent that it exists.

Step 4. Identification of Architectural Drivers. The facilitators will share their list of key architectural drivers that they assembled during steps 2 and 3, and ask the stakeholders for clarifications, additions, deletions, and corrections. The idea is to reach a consensus on a distilled list of architectural drivers that includes overall requirements, business drivers, constraints, and quality attributes.

Step 5. Scenario Brainstorming. Each stakeholder expresses a scenario representing his or her concerns with respect to the system. Facilitators ensure that each scenario has an explicit stimulus and response. The facilitators ensure that at least one representative scenario exists for each architectural driver listed in step 4.

Step 6. Scenario Consolidation. After the scenario brainstorming, similar scenarios are consolidated where reasonable. Facilitators ask stakeholders to identify those scenarios that are very similar in content. Scenarios that are similar are merged, as long as the people who proposed them agree and feel that their scenarios will not be diluted in the process. Consolidation helps to prevent votes from being spread across several scenarios that are

expressing the same concern. Consolidating almost-alike scenarios assures that the underlying concern will get all of the votes it is due.

Step 7. Scenario Prioritization. Prioritization of the scenarios is accomplished by allocating each stakeholder a number of votes equal to 30 percent of the total number of scenarios generated after consolidation. Stakeholders can allocate any number of their votes to any scenario or combination of scenarios. The votes are counted, and the scenarios are prioritized accordingly.

Step 8. Scenario Refinement. After the prioritization, the top scenarios are refined and elaborated. Facilitators help the stakeholders put the scenarios in the six-part scenario form of source-stimulus-artifact-environment-response-response measure that we described in Chapter 4. As the scenarios are refined, issues surrounding their satisfaction will emerge. These are also recorded. Step 8 lasts as long as time and resources allow.

16.3. GATHERING ASRS BY UNDERSTANDING THE BUSINESS GOALS

Business goals are the *raison d'être* for building a system. No organization builds a system without a reason; rather, the organization's leaders want to further the mission and ambitions of their organization and themselves. Common business goals include making a profit, of course, but most organizations have many more concerns than simply profit, and in other organizations (e.g., nonprofits, charities, governments), profit is the farthest thing from anyone's mind.

Business goals are of interest to architects because they often are the precursor or progenitor of requirements that may or may not be captured in a requirements specification but whose achievement (or lack) signals a successful (or less than successful) architectural design. Business goals frequently lead directly to ASRs.

There are three possible relationships between business goals and an architecture:

1. *Business goals often lead to quality attribute requirements.* Or to put it another way, every quality attribute requirement—such as user-visible response time or platform flexibility or ironclad security or any of a dozen other needs—should originate from some higher purpose that can be described in terms of added value. If we ask, for example, "*Why* do you want this system to have a really fast response time?", we might hear that this will differentiate the product from its competition and let the developing organization capture market share; or that this will make the soldier a more effective warfighter, which is the mission of the acquiring organization; or other reasons having to do with the satisfaction of some business goal.

2. *Business goals may directly affect the architecture without precipitating a quality attribute requirement at all.* In Chapter 3 we told the story of the architect who designed a system without a database until the manager informed him that the database team needed work. The architecture was importantly affected without any relevant quality attribute requirement.

3. *No influence at all.* Not all business goals lead to quality attributes. For example, a business goal to "reduce cost" may be realized by lowering the facility's thermostats in the winter or reducing employees' salaries or pensions.

Figure 16.1 illustrates the major points just described. In the figure, the arrows mean "leads to." The solid arrows are the ones highlighting relationships of most interest to architects.

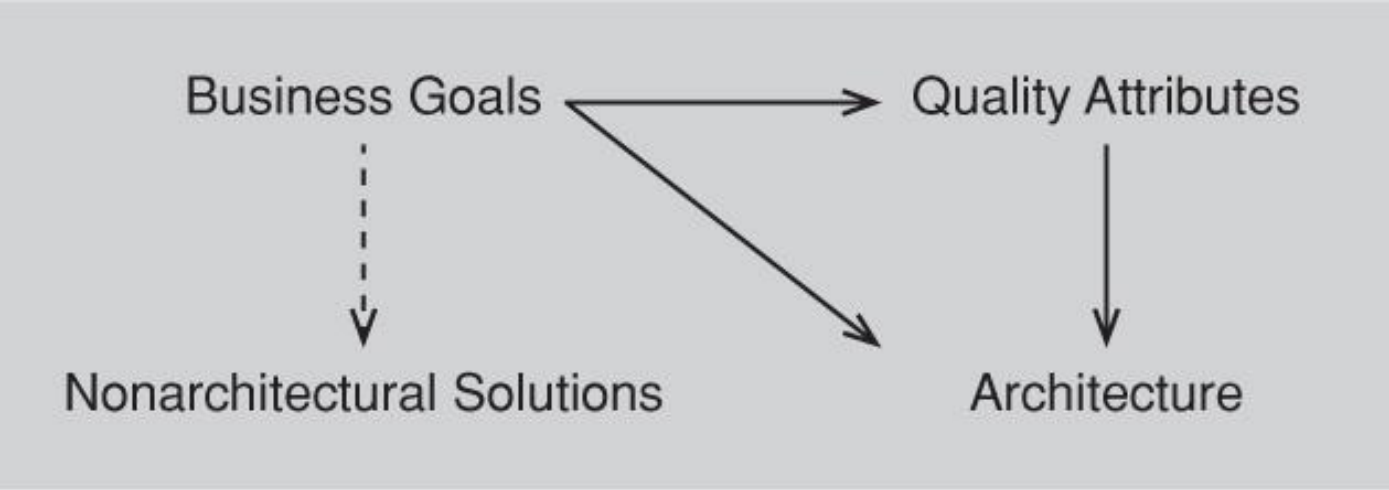

Figure 16.1. Some business goals may lead to quality attribute requirements (which lead to architectures), or lead directly to architectural decisions, or lead to nonarchitectural solutions.

Architects often become aware of an organization's business and business goals via osmosis—working, listening, talking, and soaking up the goals that are at work in an organization. Osmosis is not without its benefits, but more systematic ways are possible. We describe one such way in the sidebar "A Method for Capturing Business Goals."

A Categorization of Business Goals

Business goals are worth capturing explicitly. This is because they often imply ASRs that would otherwise go undetected until it is too late or too expensive to address them. Capturing business goals is well served by having a set of candidate business goals handy to use as conversation starters. If you know that many businesses want to gain market share, for instance, you can use that to engage the right stakeholders in your organization to ask, "What are our ambitions about market share for this product, and how could the architecture contribute to meeting them?"

Our research in business goals has led us to adopt the categories shown in Table 16.2. These categories can be used as an aid to brainstorming and elicitation. By employing the list of categories, and asking the stakeholders about possible business goals in each category, some assurance of coverage is gained.

Table 16.2. A List of Standard Business Goal Categories

1. Contributing to the growth and continuity of the organization
2. Meeting financial objectives
3. Meeting personal objectives
4. Meeting responsibility to employees
5. Meeting responsibility to society
6. Meeting responsibility to state
7. Meeting responsibility to shareholders
8. Managing market position
9. Improving business processes
10. Managing the quality and reputation of products
11. Managing change in environmental factors

These categories are not completely orthogonal. Some business goals may fit into more than one category, and that's all right. In an elicitation method, the categories should prompt questions about the existence of organizational business goals that fall into that category. If the categories overlap, then this might cause us to ask redundant questions. This is not harmful and could well be helpful. The utility of these categories is to help identify *all* business goals, not to provide a taxonomy.

1. *Contributing to the growth and continuity of the organization.* How does the system being developed contribute to the growth and continuity of the organization? In one experience using this business goal category, the system being developed was the sole reason for the existence of the organization. If the system was not successful, the organization would cease to exist. Other topics that might come up in this category deal with market share, product lines, and international sales.

2. *Meeting financial objectives.* This category includes revenue generated or saved by the system. The system may be for sale, either in standalone form or by providing a service, in which case it generates revenue. The system may be for use in an internal process, in which case it should make those processes more effective or more efficient. Also in this category is the cost of development, deployment, and operation of the system. But this category can also include financial objectives of individuals: a manager hoping for a raise, for example, or a shareholder expecting a dividend.

3. *Meeting personal objectives.* Individuals have various goals associated with the construction of a system. They may range from "I want to enhance my reputation by the success of this system" to "I want to learn new technologies" to "I want to gain experience with a different portion of the development process than in the past." In any case, it is possible that technical decisions are influenced by personal objectives.

4. *Meeting responsibility to employees.* In this category, the employees in question are usually those employees involved in development or those involved in operation. Responsibility to employees involved in development might include ensuring that certain types of employees have a role in the development of this system, or it might include providing employees the opportunities to learn new skills. Responsibility to

employees involved in operating the system might include safety, workload, or skill considerations.

5. *Meeting responsibility to society*. Some organizations see themselves as being in business to serve society. For these organizations, the system under development is helping them meet those responsibilities. But all organizations must discharge a responsibility to society by obeying relevant laws and regulations. Other topics that might come up under this category are resource usage, "green computing," ethics, safety, open source issues, security, and privacy.

6. *Meeting responsibility to state*. Government systems, almost by definition, are intended to meet responsibility to a state or country. Other topics that might come up in this category deal with export controls, regulatory conformance, or supporting government initiatives.

7. *Meeting responsibility to shareholders*. There is overlap between this category and the financial objectives category, but additional topics that might come up here are liability protection and certain types of regulatory conformance such as, in the United States, adherence to the Sarbanes-Oxley Act.

8. *Managing market position*. Topics that might come up in this category are the strategy used to increase or hold market share, various types of intellectual property protection, or the time to market.

9. *Improving business processes*. Although this category partially overlaps with meeting financial objectives, reasons other than cost reduction exist for improving business processes. It may be that improved business processes enable new markets, new products, or better customer support.

10. *Managing the quality and reputation of products*. Topics that might come up in this category include branding, recalls, types of potential users, quality of existing products, and testing support and strategies.

11. *Managing change in environmental factors*. As we said in Chapter 3, the business context for a system might change. This item is intended to encourage the stakeholders to consider what might change in the business goals for a system.

Expressing Business Goals

How will you write down a business goal once you've learned it? Just as for quality attributes, a scenario makes a convenient, uniform, and clarifying way to express business goals. It helps ensure that all business goals are expressed clearly, in a consistent fashion, and contain sufficient information to enable their shared understanding by relevant stakeholders. Just as a quality attribute scenario adds precision and meaning to an otherwise vague need for, say, "modifiability," a business goal scenario will add precision and meaning to a desire to "meet financial objectives."

Our business goal scenario template has seven parts. They all relate to the system under development, the identity of which is implicit. The parts are these:

1. *Goal-source*. These are the people or written artifacts providing the goal.

2. *Goal-subject*. These are the stakeholders who own the goal and wish it to be true. Each stakeholder might be an individual or (in the case of a goal that has no one owner and has been assimilated into an organization) the organization itself. If the

business goal is, for example, “Maximize dividends for the shareholders,” who is it that cares about that? It is probably not the programmers or the system’s end users (unless they happen to own stock). Goal-subjects can and do belong to different organizations. The developing organization, the customer organizations, subcontractors, vendors and suppliers, standards bodies, regulatory agencies, and organizations responsible for systems with which ours must interact are all potential goal-subjects.

3. *Goal-object*. These are the entities to which the goal applies. “Object” is used in the sense of the object of a verb in a sentence. All goals have goal-objects: we want something to be true about something (or someone) that (or whom) we care about. For example, for goals we would characterize as furthering one’s self-interest, the goal-object can be “myself or my family.” For some goals the goal-object is clearly the development organization, but for some goals the goal-object can be more refined, such as the rank-and-file employees of the organization or the shareholders of the organization. Table 16.3 is a representative cross-section of goal-objects. Goal-objects in the table start small, where the goal-object is a single individual, and incrementally grow until the goal-object is society at large.

Table 16.3. Business Goals and Their Goal-Objects

Goal-Object	Business Goals That Often Have This Goal-Object	Remarks
Individual	Personal wealth, power, honor/face/reputation, game and gambling spirit, maintain or improve reputation (personal), family interests	The individual who has these goals has them for him/herself or his/her family.
System	Manage flexibility, distributed development, portability, open systems/standards, testability, product lines, integrability, interoperability, ease of installation and ease of repair, flexibility/configurability, performance, reliability/availability, ease of use, security, safety, scalability/extendibility, functionality, system constraints, internationalization, reduce time to market	These can be goals for a system being developed or acquired. The list applies to systems in general, but the quantification of any one item likely applies to a single system being developed or acquired.
Portfolio	Reduce cost of development, cost leadership, differentiation, reduce cost of retirement, smooth transition to follow-on systems, replace legacy systems, replace labor with automation, diversify operational sequence, eliminate intermediate stages, automate tracking of business events, collect/communicate/retrieve operational knowledge, improve decision making, coordinate across distance, align task and process, manage on basis of process measurements, operate effectively within the competitive environment, the technological environment, or the customer environment Create something new, provide the best quality products and services possible, be the leading innovator in the industry	These goals live on the cusp between an individual system and the entire organization. They apply either to a single system or to an organization’s entire portfolio that the organization is building or acquiring to achieve its goals.
Organization’s Employees	Provide high rewards and benefits to employees, create a pleasant and friendly workplace, have satisfied employees, fulfill responsibility toward employees, maintain jobs of workforce on legacy systems	Before we get to the organization as a whole, there are some goals aimed at specific subsets of the organization.
Organization’s Shareholders	Maximize dividends for the shareholders	
Organization	Growth of the business, continuity of the business, maximize profits over the short run, maximize profits over the long run, survival of the organization, maximize the company’s net assets and reserves, be a market leader, maximize the market share, expand or retain market share, enter new markets, maximize the company’s rate of growth, keep tax payments to a minimum, increase sales growth, maintain or improve reputation, achieve business goals through financial objectives, run a stable organization	These are goals for the organization as a whole. The organization can be a development or acquisition organization, although most were undoubtedly created with the former in mind.
Nation	Patriotism, national pride, national security, national welfare	Before we get to society at large, this goal-object is specifically limited to the goal owner’s own country.
Society	Run an ethical organization, responsibility toward society, be a socially responsible company, be of service to the community, operate effectively within social environment, operate effectively within legal environment	Some interpret “society” as “my society,” which puts this category closer to the nation goal-object, but we are taking a broader view.

4. *Environment*. This is the context for this goal. For example, there are social, legal, competitive, customer, and technological environments. Sometimes the political environment is key; this is as a kind of social factor. Upcoming technology may be a major factor.

5. *Goal*. This is any business goal articulated by the goal-source.

6. *Goal-measure.* This is a testable measurement to determine how one would know if the goal has been achieved. The goal-measure should usually include a time component, stating the time by which the goal should be achieved.

7. *Pedigree and value.* The pedigree of the goal tells us the degree of confidence the person who stated the goal has in it, and the goal's volatility and value. The value of a goal can be expressed by how much its owner is willing to spend to achieve it or its relative importance compared to other goals. Relative importance may be given by a ranking from 1 (most important) to *n* (least important), or by assigning each goal a value on a fixed scale such as 1 to 10 or high-medium-low. We combine value and pedigree into one part although it certainly is possible to treat them separately. The important concern is that both are captured.

Elements 2–6 can be combined into a sentence that reads:

For the system being developed, <goal-subject> desires that <goal-object> achieve <goal> in the context of <environment> and will be satisfied if <goal-measure>.

The sentence can be augmented by the goal's source (element 1) and the goal's pedigree and value (element 7). Some sample business goal scenarios include the following:

• For MySys, the project manager has the goal that his family's stock in the company will rise by 5 percent (as a result of the success of MySys).

• For MySys, the developing organization's CEO has the goal that MySys will make it 50 percent less likely that his nation will be attacked.

• For MySys, the portfolio manager has the goal that MySys will make the portfolio 30 percent more profitable.

• For MySys, the project manager has the goal that customer satisfaction will rise by 10 percent (as a result of the increased quality of MySys).

In many contexts, the goals of different stakeholders may conflict. By identifying the stakeholder who owns the goal, the sources of conflicting goals can be identified.

A General Scenario for Business Goals

A general scenario (see Chapter 4) is a template for constructing specific or "concrete" scenarios. It uses the generic structure of a scenario to supply a list of possible values for each non-boilerplate part of a scenario. See Table 16.4 for a general scenario for business goals.

Table 16.4. General Scenario Generation Table for Business Goals

2. Goal-subject	. . . has the goal that . . .	3. Goal-object	. . . achieves . . .	5. Goal	. . . in the context of . . .	4. Environment	. . . and will be satisfied if . . .	6. Goal-measure (examples, based on goal categories)	7. Value
Any stakeholder or stakeholder group identified as having a *legitimate* interest in the system		Individual		Contributing to the growth and continuity of the organization		Social (includes political)		Time that business remains viable	1–*n*
		System		Meeting financial objectives		Legal		Financial performance vs. objectives	1–10
		Portfolio		Meeting personal objectives		Competitive		Promotion or raise achieved in period	H-M-L
		Organization's employees		Meeting responsibility to employees		Customer		Employee satisfaction; turnover rate	Resources willing to expend
		Organization's shareholders		Meeting responsibility to society		Technological		Contribution to trade deficit/ surplus	
		Organization		Meeting responsibility to state				Stock price, dividends	
		Nation		Meeting responsibility to shareholders				Market share	
		Society		Managing market position				Time to carry out a business process	
				Improving business processes				Quality measures of products	
				Managing quality and reputation of products				Technology-related problems	
				Managing change in environmental factors				Time window for achievement	

For each of these scenarios you might want to additionally capture its source (e.g., Did this come directly from the goal-subject, a document, a third party, a legal requirement?), its volatility, and its importance.

Capturing Business Goals

Business goals are worth capturing because they can hold the key to discovering ASRs that emerge in no other context. One method for eliciting and documenting business goals is the Pedigreed Attribute eLicitation Method, or PALM. The word "pedigree" means that the business goal has a clear derivation or background. PALM uses the standard list of business goals and the business goal scenario format we described earlier.

PALM can be used to sniff out missing requirements early in the life cycle. For example, having stakeholders subscribe to the business goal of improving the quality and reputation of their products may very well lead to (for example) security, availability, and performance requirements that otherwise might not have been considered.

PALM can also be used to discover and carry along additional information about existing requirements. For example, a business goal might be to produce a product that outcompetes a rival's market entry. This might precipitate a performance requirement for, say, half-second turnaround when the rival features one-second turnaround. But if the competitor releases a new product with half-second turnaround, then what does our requirement become? A conventional requirements document will continue to carry the half-second requirement, but the goal-savvy architect will know that the real requirement is to beat the competitor, which may mean even faster performance is needed.

Finally, PALM can be used to examine particularly difficult quality attribute requirements to see if they can be relaxed. We know of more than one system where a quality attribute requirement proved quite expensive to provide, and only after great effort, money, and time were expended trying to meet it was it revealed that the requirement had no actual basis other than being someone's best guess or fond wish at the time.

16.4. CAPTURING ASRS IN A UTILITY TREE

As we have seen, ASRs can be extracted from a requirements document, captured from stakeholders in a workshop such as a QAW, or derived from business goals. It is helpful to record them in one place so that the list can be reviewed, referenced, used to justify design decisions, and revisited over time or in the case of major system changes.

To recap, an ASR must have the following characteristics:

• *A profound impact on the architecture.* Including this requirement will very likely result in a different architecture than if it were not included.

• *A high business or mission value.* If the architecture is going to satisfy this requirement—potentially at the expense of not satisfying others—it must be of high value to important stakeholders.

Using a single list can also help evaluate each potential ASR against these criteria, and to make sure that no architectural drivers, stakeholder classes, or business goals are lacking ASRs that express their needs.

A Method for Capturing Business Goals

PALM is a seven-step method, nominally carried out over a day and a half in a workshop attended by architects and stakeholders who can speak to the business goals of the organizations involved. The steps are these:

1. *PALM overview presentation*. Overview of PALM, the problem it solves, its steps, and its expected outcomes.

2. *Business drivers presentation*. Briefing of business drivers by project management. What are the goals of the customer organization for this system? What are the goals of the development organization? This is normally a lengthy discussion that allows participants to ask questions about the business goals as presented by project management.

3. *Architecture drivers presentation*. Briefing by the architect on the driving business and quality attribute requirements: the ASRs.

4. *Business goals elicitation*. Using the standard business goal categories to guide discussion, we capture the set of important business goals for this system. Business goals are elaborated and expressed as scenarios. We consolidate almost-alike business goals to eliminate duplication. Participants then prioritize the resulting set to identify the most important goals.

5. *Identification of potential quality attributes from business goals*. For each important business goal scenario, participants describe a quality attribute that (if architected into the system) would help achieve it. If the QA is not already a requirement, this is recorded as a finding.

6. *Assignment of pedigree to existing quality attribute drivers*. For each architectural driver named in step 3, we identify which business goals it is there to support. If none, that's recorded as a finding. Otherwise, we establish its pedigree by asking for the source of the quantitative part. For example: Why is there a 40-millisecond performance requirement? Why not 60 milliseconds? Or 80 milliseconds?

7. *Exercise conclusion*. Review of results, next steps, and participant feedback.

Architects can use a construct called a *utility tree* for all of these purposes. A utility tree begins with the word "utility" as the root node. Utility is an expression of the overall "goodness" of the system. We then elaborate this root node by listing the major quality attributes that the system is required to exhibit. (We said in Chapter

4 that quality attribute names by themselves were not very useful. Never fear: we are using them only as placeholders for subsequent elaboration and refinement!)

Under each quality attribute, record a specific refinement of that QA. For example, performance might be decomposed into “data latency” and “transaction throughput.” Or it might be decomposed into “user wait time” and “time to refresh web page.” The refinements that you choose should be the ones that are relevant to your system. Under each refinement, record the appropriate ASRs (usually expressed as QA scenarios).

Some ASRs might express more than one quality attribute and so might appear in more than one place in the tree. That is not necessarily a problem, but it could be an indication that the ASR tries to cover too much diverse territory. Such ASRs may be split into constituents that each attach to smaller concerns.

Once the ASRs are recorded and placed in the tree, you can now evaluate them against the two criteria we listed above: the business value of the candidate ASR and the architectural impact of including it. You can use any scale you like, but we find that a simple “H” (high), “M” (medium), and “L” (low) suffice for each criterion.

For business value, High designates a must-have requirement, Medium is for a requirement that is important but would not lead to project failure were it omitted. Low describes a nice requirement to have but not something worth much effort.

For architectural impact, High means that meeting this ASR will profoundly affect the architecture. Medium means that meeting this ASR will somewhat affect the architecture. Low means that meeting this candidate ASR will have little effect on the architecture.

Table 16.5 shows a portion of a sample utility tree drawn from a health care application called Nightingale. Each ASR is labeled with a pair of “H,” “M,” and “L” values indicating (a) the ASR’s business value and (b) its effect on the architecture.

Table 16.5. Tabular Form of the Utility Tree for the Nightingale ATAM Exercise

Quality Attribute	Attribute Refinement	ASR
Performance	Transaction response time	A user updates a patient's account in response to a change-of-address notification while the system is under peak load, and the transaction completes in less than 0.75 second. (H,M)
		A user updates a patient's account in response to a change-of-address notification while the system is under double the peak load, and the transaction completes in less than 4 seconds. (L,M)
	Throughput	At peak load, the system is able to complete 150 normalized transactions per second. (M,M)
Usability	Proficiency training	A new hire with two or more years' experience in the business becomes proficient in Nightingale's core functions in less than 1 week. (M,L)
		A user in a particular context asks for help, and the system provides help for that context, within 3 seconds. (H,M)
	Normal operations	A hospital payment officer initiates a payment plan for a patient while interacting with that patient and completes the process without the system introducing delays. (M,M)
Configurability	User-defined changes	A hospital increases the fee for a particular service. The configuration team makes the change in 1 working day; no source code needs to change. (H,L)
Maintainability	Routine changes	A maintainer encounters search- and response-time deficiencies, fixes the bug, and distributes the bug fix with no more than 3 person-days of effort. (H,M)
		A reporting requirement requires a change to the report-generating metadata. Change is made in 4 person-hours of effort. (M,L)
	Upgrades to commercial components	The database vendor releases a new version that must be installed in less than 3 person-weeks. (H,M)
Extensibility	Adding new product	A product that tracks blood bank donors is created within 2 person-months. (M,M)
Security	Confidentiality	A physical therapist is allowed to see that part of a patient's record dealing with orthopedic treatment but not other parts nor any financial information. (H,M)
	Integrity	The system resists unauthorized intrusion and reports the intrusion attempt to authorities within 90 seconds. (H,M)
Availability	No downtime	The database vendor releases new software, which is hot-swapped into place, with no downtime. (H,L)
		The system supports 24/7 web-based account access by patients. (L,L)

Once you have a utility tree filled out, you can use it to make important checks. For instance:

• A QA or QA refinement without any ASR is not necessarily an error or omission that needs to be rectified, but it is an indication that attention should be paid to finding out for sure if there are unrecorded ASRs in that area.

• ASRs that rate a (H,H) rating are obviously the ones that deserve the most attention from you; these are the most significant of the significant requirements. A very large number of these might be a cause for concern about whether the system is achievable.

• Stakeholders can review the utility tree to make sure their concerns are addressed. (An alternative to the organization we have described here is to use stakeholder roles rather than quality attributes as the organizing rule under "Utility.")

16.5. TYING THE METHODS TOGETHER

How should you employ requirements documents, stakeholder interviews, Quality Attribute Workshops, PALM, and utility trees in concert with each other?

As for most complex questions, the answer to this one is "It depends." If you have a requirements process that gathers, identifies, and prioritizes ASRs, then use that and consider yourself lucky.

If you feel your requirements fall short of this ideal state, then you can bring to bear one or more of the other approaches. For example, if nobody has captured the business goals behind the system you're building, then a PALM exercise would be a good way to ensure that those goals are represented in the system's ASRs.

If you feel that important stakeholders have been overlooked in the requirements-gathering process, then it will probably behoove you to capture their concerns through interviews. A Quality Attribute Workshop is a structured method to do that and capture their input.

Building a utility tree is a good way to capture ASRs along with their prioritization—something that many requirements processes overlook.

Finally, you can blend all the methods together: PALM makes an excellent "subroutine call" from a Quality Attribute Workshop for the step that asks about business goals, and a quality attribute utility tree makes an excellent repository for the scenarios that are the workshop's output.

It is unlikely, however, that your project will have the time and resources to support this do-it-all approach. Better to pick the approach that fills in the biggest gap in your existing requirements: stakeholder representation, business goal manifestation, or ASR prioritization.

16.6. SUMMARY

Architectures are driven by architecturally significant requirements: requirements that will have profound effects on the architecture. Architecturally significant requirements may be captured from requirements documents, by interviewing stakeholders, or by conducting a Quality Attribute Workshop.

In gathering these requirements, we should be mindful of the business goals of the organization. Business goals can be expressed in a common, structured form and

represented as scenarios. Business goals may be elicited and documented using a structured facilitation method called PALM.

A useful representation of quality attribute requirements is in a utility tree. The utility tree helps to capture these requirements in a structured form, starting from coarse, abstract notions of quality attributes and gradually refining them to the point where they are captured as scenarios. These scenarios are then prioritized, and this prioritized set defines your "marching orders" as an architect.

16.7. FOR FURTHER READING

PALM can be used to capture the business goals that conform to a business goal viewpoint; that is, you can use PALM to populate a business goal view of your system, using the terminology of ISO Standard 42010. We discuss this in *The Business Goals Viewpoint* [Clements 10c]. Complete details of PALM can be found in CMU/SEI-2010-TN-018, *Relating Business Goals to Architecturally Significant Requirements for Software Systems* [Clements 10b].

The Open Group Architecture Framework, available at www.opengroup.org/togaf/, provides a very complete template for documenting a business scenario that contains a wealth of useful information. Although we believe architects can make use of a lighter-weight means to capture a business goal, it's worth a look.

The definitive reference source for the Quality Attribute Workshop is [Barbacci 03].

The term *architecturally significant requirement* was created by the Software Architecture Review and Assessment (SARA) group [Obbink 02].

When dealing with systems of systems (SoS), the interaction and handoff between the systems can be a source of problems. The Mission Thread Workshop and Business Thread Workshop focus on a single thread of activity within the overall SoS context and identify potential problems having to do with the interaction of the disparate systems. Descriptions of these workshops can be found at [Klein 10] and [Gagliardi 09].

16.8. DISCUSSION QUESTIONS

1. Interview representative stakeholders for your business's or university's expense recovery system. Capture the business goals that are motivating the system. Use the seven-part business goal scenario outline given in Section 16.3.

2. Draw a relation between the business goals you uncovered for the previous question and ASRs.

3. Consider an automated teller machine (ATM) system. Attempt to apply the 11 categories of business goals to that system and infer what goals might have been held by various stakeholders involved in its development.

4. Create a utility tree for the ATM system above. (Interview some of your friends and colleagues if you like, to have them contribute quality attribute considerations and scenarios.) Consider a minimum of four different quality attributes. Ensure that the scenarios that you create at the leaf nodes have explicit responses and response measures.

5. Restructure the utility tree given in Section 16.4 using stakeholder roles as the organizing principle. What are the benefits and drawbacks of the two representations?

6. Find a software requirements specification that you consider to be of high quality. Using colored pens (real ones if the document is printed, virtual ones if the document is online), color red all the material that you find completely irrelevant to a software architecture for that system. Color yellow all of the material that you think *might* be relevant, but not without further discussion and elaboration. Color green all of the material that you are certain is architecturally significant. When you're done, every part of the document that's not white space should be red, yellow, or green. Approximately what percentage of each color did your document end up being? Do the results surprise you?